

# Conversa — How everything works

A reference for the architecture and each subsystem. For a click-by-click test script, see TESTING-GUIDE.md.

## 1. Stack & big picture

| Layer            | Choice                                                   |
|------------------|----------------------------------------------------------|
| Framework        | Next.js 16 (App Router, React 19, Server Components ...) |
| Language         | TypeScript (strict)                                      |
| Styling          | Tailwind CSS v4 (CSS-first tokens in globals.css)        |
| UI               | Hand-built shadcn-style primitives in src/components...  |
| Data / Auth      | Supabase (Postgres, Auth, Realtime, Storage)             |
| Server state     | React Server Components + Server Actions                 |
| Client state     | TanStack Query where needed; local component state e...  |
| Forms/validation | React Hook Form + Zod (and Zod in server actions)        |
| Charts           | Recharts                                                 |
| Flow builder     | dnd-kit                                                  |

Conversa is **multi-tenant**: every row belongs to an [organisation](#), and Postgres **Row-Level Security (RLS)** enforces that users only ever read/write data for organisations they are an active member of. The UI additionally gates features by role (capabilities), but RLS is the real security boundary.

## 2. Directory map

```
src/
  app/
    (marketing)/      Public site (home). Own layout with nav + footer.
    (auth)/           login, register, forgot-password, reset-password
    onboarding/       Create-organisation wizard (step 1 functional)
    auth/callback/route.ts  Exchanges email/reset codes for a session
    app/              Authenticated product; layout loads app context
      inbox/          Shared inbox (4-column) + realtime
      contacts/ reports/ ai-agents/ knowledge/ chatbots/[id]/
      broadcasts/ integrations/ settings/ notifications/
  components/
    ui/               Button, Input, Card, Badge, Avatar, Menu, Skeleton...
    app/              PrimarySidebar, Topbar, PageHeader, EmptyState, nav
    inbox/            Composer, ConversationActions, meta, realtime
    chatbots/         FlowBuilder, NodeCard, SettingsPanel, Simulator
    reports/          Recharts wrappers
    marketing/ brand/
```

|                      |                                                      |
|----------------------|------------------------------------------------------|
| lib/                 |                                                      |
| supabase/            | client (browser), server, admin, middleware, types   |
| auth/                | context (session/org), roles (capabilities), actions |
| data/                | conversation queries + server actions                |
| chatbots/            | types, geometry, validate, simulate, actions         |
| onboarding/          | validations/                                         |
| supabase/migrations/ | 0001-0009 ordered SQL (schema, RLS, realtime)        |
| scripts/             | apply-migrations, seed, gen-types, emit-test-message |
| docs/                | this folder                                          |

### 3. Request lifecycle & auth

- **Middleware** (`src/middleware.ts` → `lib/supabase/middleware.ts`) runs on every request, refreshes the Supabase session cookie, and redirects unauthenticated users away from `/app/*` and `/onboarding`. (It no-ops before Supabase env vars are set so the app is still inspectable.)
- **Server Components** read data with a `**request-scoped Supabase server client**` (`lib/supabase/server.ts`) that carries the user's cookies, so all queries run **as the signed-in user under RLS**.
- ``getAppContext()`` (`lib/auth/context.ts`, React-cached per request) resolves the user, their `user_profiles` row, all active memberships, and the **active organisation** (from the `conversa_org` cookie, else the default membership). It redirects to `/login` when signed out and `/onboarding` when the user has no organisation.
- **Mutations** are **Server Actions** ("use server"). They re-resolve the context, perform the write under RLS, write an `audit_logs` row where relevant, and call `revalidatePath` so the affected Server Components re-render.

#### Auth flows

`lib/auth/actions.ts` wraps Supabase Auth: `signInAction`, `signUpAction`, `signOutAction`, `forgotPasswordAction`, `resetPasswordAction`. Email confirmation and password-reset links land on `/auth/callback`, which exchanges the code for a session and redirects onward.

### 4. Multi-tenancy & RLS (the security model)

- Every tenant table has an `organisation_id`.
- Membership lives in `organisation_members` (`user_id`, `organisation_id`, `role`).
- Helper functions (SECURITY DEFINER, in `0002`):
  - `is_org_member(org)` — is the caller an active member?

- `has_org_role(org, roles[]) / can_manage_org(org)` — role checks.
- `0008_rls_policies.sql`:
  - A loop enables RLS and adds a generic `org_isolation` policy (`using/ with check is_org_member(organisation_id)`) to **every** table that has an `organisation_id`.
  - Junction/child tables without that column (e.g. `contact_tags`, `conversation_participants`, `ticket_status_history`) are gated by an `EXISTS` join to their parent.
  - `organisations`, `user_profiles`, `roles`, and reference tables (`permissions`, `subscription_plans`) get bespoke policies.
  - **Service role** (used only by `scripts/seed.ts` and — in future — inbound webhooks / the public widget endpoint) bypasses RLS by design and is never exposed to the browser.

## Roles → capabilities (UI gating)

`lib/auth/roles.ts` maps the 7 roles (Platform Admin, Owner, Org Admin, Support Manager, Support Agent, Marketing, Reporting) to capability keys (`inbox.view`, `automations.manage`, `broadcasts.manage`, ...). `PrimarySidebar` shows only the modules a role can access; server actions still re-check on the server.

---

## 5. Shared inbox

- **Data** (`lib/data/conversations.ts`): `listConversations(org, user, view)` applies the selected view (mine / unassigned / waiting / resolved / AI-handled / spam / all) as query filters; `getViewCounts` returns badge counts; `getConversation` loads a thread with messages, notes and contact.
- **Page** (`app/app/inbox/page.tsx`) is a Server Component reading `?view=` and `?c=` from the URL, rendering the four columns:
 

views · conversation list · workspace (timeline + composer) · details.
- **Actions** (`lib/data/conversation-actions.ts`): `sendReplyAction`, `addNoteAction`, `setStatusAction`, `setPriorityAction`, `assignToMeAction` — each writes under RLS and revalidates.
- **Composer** (client) supports Reply, Internal note, **AI draft (demo)**, and “Send & resolve”.

## Realtime

-

`0009_realtime.sql` adds `messages`, `conversations`, `internal_notes` to the `supabase_realtime` publication with `REPLICA IDENTITY FULL`.

- `components/inbox/realtime.tsx` subscribes (browser client) to `postgres_changes` for those tables filtered by `organisation_id`, sets the Realtime JWT so **RLS applies to the stream**, and debounces a `router.refresh()` on any change — so the server-rendered inbox re-queries and the list order, previews, counts and open thread all update live. A green
  - **Live** indicator shows connection state.
  - Test it with `npx tsx scripts/emit-test-message.ts "..."` while the inbox is open (see the testing guide).
- 

## 6. AI (demo mode)

- `AI_PROVIDER=demo` by default. Demo responses are **\*\*deterministic and clearly labelled ("AI · demo mode") and are never\*\*** presented as coming from a live model. In the inbox, "AI draft" inserts a labelled draft you must review before sending; AI-handled conversations render an "AI agent · demo" bubble and an `ai_runs` row is recorded.
  - The schema (`ai_agents`, `ai_agent_versions`, `ai_agent_sources`, `ai_runs`, `ai_feedback`) plus knowledge tables model a grounded-agent design: answer only from approved sources, record every run with a confidence score, and escalate.
  - To enable a live provider later: set `AI_PROVIDER=anthropic` and `ANTHROPIC_API_KEY`, then implement the provider adapter behind the same interface. Demo mode remains the fallback when no key is present.
- 

## 7. Visual chatbot builder

- **Model** (`lib/chatbots/types.ts`): `FlowDefinition = { nodes, edges }`. 18 node types with a catalogue (`NODE_DEFS`) describing icon, colour, category, output handles and default data.
- **Editor** (`components/chatbots/flow-builder.tsx`): a canvas with pan (drag background), zoom, minimap and dotted grid. Nodes are dragged with **dnd-kit** (delta ÷ scale committed to `position`). Edges are created by **\*\*click a source handle → click a target node\*\***, drawn as SVG bezier curves with arrowheads; click an edge to delete it. Branch handles exist for Condition (True/False), Multiple Choice (per option) and Business Hours (Open/Closed).
- **Validation** (`lib/chatbots/validate.ts`): missing/duplicate Start, dangling edges, dead-ends and unreachable nodes.

- **Simulator** (`lib/chatbots/simulate.ts` + `components/chatbots/simulator.tsx`):

a pure, deterministic interpreter that runs the flow as a chat — great for testing branches without publishing.

- **Persistence** (`lib/chatbots/actions.ts`): draft `definition` is saved on the `chatbot_flows` row; **Publish** validates, writes a `chatbot_versions` row and bumps `current_version`; **Duplicate** clones the flow. Definitions are versioned JSON with relational ownership/status/audit.

## 7b. Automation rules

- **Model**: ``automation_rules` (trigger_type, trigger_config, conditions[], actions[], status, position)` with an `automation_runs`` history table.
- **Catalogue** (`lib/automations/catalogue.ts`): 10 triggers, 7 condition fields × 4 operators, and 12 action types.
- **List** (`/app/automations`): rules ordered by `position`, **drag-to-reorder** (dnd-kit sortable, persisted), enable/disable switch, delete.
- **Editor** (`/app/automations/[id]`): a **When** → **If** → **Then** form — pick a trigger (+inline config), add AND-ed conditions (field/operator/value), and add actions (each with its own parameter). Saved via a server action.
- **Test** (`lib/automations/evaluate.ts` + `testRuleAction`): evaluates the rule's conditions against your most recent conversation, records an `automation_runs` row (success/skipped), and shows which actions *would* run.
- The live event-driven engine (firing rules automatically on events, with the `automation_runs.depth` loop-protection column) is a labelled build-out; ordering, storage, evaluation and history are in place.

## 7c. Broadcast composer

- **Audience** (`lib/broadcasts/audience.ts`): `resolveAudience` resolves the eligible recipients for a channel + segment (tags / company / search), **enforcing consent** (`consent_status = opted_in`) and the *\*\*suppression list\*\**, and requiring a usable channel identifier. It returns the eligible contacts plus excluded counts (no consent / suppressed / no address).
- **Personalisation** (`lib/broadcasts/personalize.ts`): `{{first_name}}`, `{{last_name}}`, `{{company}}` are detected and substituted per recipient; the composer shows a live preview.
- **Composer** (`/app/broadcasts/[id]`): message (channel, optional approved template, body + variable chips + preview) · audience (tag filters, consent toggle, **Preview audience** with live counts) · schedule (send now / schedule,

frequency cap) · a right rail for **send test**, **approval gate**, and **dispatch**.

- **Dispatch** (`dispatchBroadcastAction`): requires approval, materialises the eligible audience into `broadcast_recipients` (idempotent upsert per broadcast+contact), then schedules or **simulates delivery** (demo — no live provider is contacted) and records per-recipient statuses.
- **Delivery tracking**: metrics (queued/sent/delivered/read/replied/failed/opted-out) are aggregated from `broadcast_recipients` and shown as a stacked bar + legend.

## 8. Data model (overview)

~60 tables across `0001–0007`, grouped:

- **Tenancy**: `organisations`, `organisation_members`, `user_profiles`, `roles`, `permissions`, `teams`, `team_members`, `agent_availability`
- **Channels/contacts**: `inboxes`, `channels`, `channel_connections`, `contacts`, `contact_channels`, `contact_custom_fields`, `contact_field_values`, `tags`, `contact_tags`
- **Conversations**: `conversations`, `conversation_participants`, `conversation_tags`, `messages`, `message_attachments`, `message_status_events`, `internal_notes`, `mentions`, `tickets` (+tags/assignments/status\_history), `saved_replies`, `saved_views`
- **AI/knowledge**: `ai_agents` (+versions/sources), `ai_runs`, `ai_feedback`, `knowledge_collections/articles` (+versions)/`documents/chunks/gaps`
- **Automation/bots**: `chatbot_flows` (+versions/runs), `automation_rules` (+runs)
- **Outbound/platform**: `broadcasts` (+recipients), `message_templates`, `consent_records`, `suppression_entries`, `sla_policies`, `business_hours`, `notifications`, `webhook_endpoints`, `webhook_events`, `integration_logs`, `audit_logs`, `subscription_plans`, `organisation_subscriptions`, `usage_records`

Conventions: UUID PKs, `created_at/updated_at` (auto-touched by a trigger), FKs with sensible `on delete`, useful indexes (incl. trigram search on contacts and knowledge), soft-delete via `deleted_at` where appropriate, and a `webhook_events.idempotency_key` unique constraint for idempotent inbound events.

---

## 9. Channels (adapter pattern)

Channels are modelled generically (`channel_type` enum: whatsapp, email, web\_chat, messenger, instagram, sms). Each `channel` has a `channel_connection` holding **non-secret** config and a `secret_ref` (never the secret itself).

Unconnected channels run a **demo adapter** (clearly labelled). Live adapters (WhatsApp Business Cloud API, email via Resend, Meta Messenger/Instagram) plug in behind the same shape with webhook endpoints + signature verification. WhatsApp uses the **official Meta Cloud API** architecture — no unofficial automation.

---

## 10. Environment & scripts

See `.env.example` for all variables. Key ones:

`NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY` (public),

`SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_DB_URL` (server-only secrets),

`AI_PROVIDER` (demo by default).

| Command                        | Purpose                                            |
|--------------------------------|----------------------------------------------------|
| <code>npm run dev</code>       | Dev server                                         |
| <code>npm run build</code>     | Production build                                   |
| <code>npm run typecheck</code> | <code>tsc --noEmit</code>                          |
| <code>npm run lint</code>      | ESLint                                             |
| <code>npm run db:push</code>   | Apply supabase/migrations/*.sql (idempotent)       |
| <code>npm run db:seed</code>   | Seed the demo org (clears + recreates it)          |
| <code>npm run db:types</code>  | Generate typed DB definitions from the live schema |

> **Windows note:** on this machine `npm` intermittently drops from the shell

> PATH. If `npm run dev` fails with “npm not found”, launch Next directly:

> `node node_modules/next/dist/bin/next dev`.

---

## 11. What's live vs. still a build-out

**Fully working:** design system · auth (register/login/verify/reset) · onboarding org creation · org switching & role-gated nav · shared inbox (reply/note/AI-draft/assign/priority/resolve/reopen) · **inbox realtime** · contacts table · reports (real metrics + charts) · **chatbot builder** (canvas/validate/test/save/publish/version) · **automation-rule builder** (triggers/conditions/actions, reorder, test, run history) · **broadcast composer** (consent-enforced audience, personalisation, approval, scheduling, delivery tracking) · data-backed lists for AI agents, knowledge, integrations, settings.

**Honest build-outs (labelled in the UI):** the live event-driven automation engine (rules are built, ordered, evaluated and tested now) · live message delivery to providers (broadcasts record recipients and simulate delivery in demo mode) · live channel adapters + inbound webhooks · CSV import/merge · knowledge

document upload & re-indexing · full settings subpages · presence/typing indicators · Vitest/Playwright suites.

Having security building blocks (RLS, audit logs, consent/suppression tables) does **not** by itself make the software compliant or certified.